Prometheus

What is Prometheus?

- Prometheus is an open-source monitoring and alerting system that collects and stores time-series metrics from various sources.

What is Prometheus?

- Prometheus is an open-source monitoring and alerting system that collects and stores time-series metrics from various sources.

- For example:
  - System: A Linux server can expose CPU, memory, and disk usage metrics via the Node Exporter, which Prometheus scrapes regularly.

  - Application: A web application (e.g., Flask or Spring Boot app) can expose performance metrics like request rate or error count on a /metrics endpoint, which Prometheus collects and visualizes.

What is Prometheus?

- Prometheus is an open-source monitoring and alerting system that collects and stores time-series metrics from various sources.

- For example:
  - System: A Linux server can expose CPU, memory, and disk usage metrics via the Node Exporter, which Prometheus scrapes regularly.

  - Application: A web application (e.g., Flask or Spring Boot app) can expose performance metrics like request rate or error count on a /metrics endpoint, which Prometheus collects and visualizes.

- Prometheus was originally designed to monitor highly dynamic containerized environments such as Kubernetes and Docker Swarm. However, it is also well-suited for traditional, non-containerized infrastructures where applications run directly on bare-metal or virtual servers.

- Over the years, Prometheus has become the go-to monitoring tool in the world of containers and microservices.

Where and why is Prometheus used?

- Prometheus is used in IT infrastructure and cloud environments (like Kubernetes, servers, or microservices) to monitor system performance, application health, and resource usage, and to trigger alerts when thresholds are breached.

Where and why is Prometheus used?

- Prometheus is used in IT infrastructure and cloud environments (like Kubernetes, servers, or microservices) to monitor system performance, application health, and resource usage, and to trigger alerts when thresholds are breached.

- Prometheus is used in:
  - Cloud environments: e.g., monitoring Kubernetes clusters for CPU and memory usage.
  - Servers: e.g., tracking Linux server performance using Node Exporter.
  - Applications: e.g., observing a Flask web app's request rate and error count via /metrics.
  - Databases: e.g., monitoring MySQL using the MySQL Exporter.

What is Exporter?

- In Prometheus, an exporter is a component that collects metrics from a system and exposes them in a format that Prometheus can scrape (usually over HTTP, at the /metrics endpoint).

What is Exporter?

- In Prometheus, an exporter is a component that collects metrics from a system and exposes them in a format that Prometheus can scrape (usually over HTTP, at the /metrics endpoint).

- Why exporters are needed:
  - Prometheus doesn't directly know how to collect metrics from every kind of application, database, or system. Exporters act as translation layers — they gather metrics from their target systems (like MySQL, Redis, Node.js, or Linux) and present them in a standardized format Prometheus understands.

What is Exporter?

- In Prometheus, an exporter is a component that collects metrics from a system and exposes them in a format that Prometheus can scrape (usually over HTTP, at the /metrics endpoint).

- Why exporters are needed:
  - Prometheus doesn't directly know how to collect metrics from every kind of application, database, or system. Exporters act as translation layers — they gather metrics from their target systems (like MySQL, Redis, Node.js, or Linux) and present them in a standardized format Prometheus understands.

- Example:
  - Let's say you want to monitor a MySQL database. Prometheus can't natively query MySQL, but:
    - The MySQL exporter connects to the MySQL server, collects metrics like query rates, connections, buffer usage, etc., and exposes them at an HTTP endpoint like: http://your-server:9104/metrics
    - Prometheus then scrapes this endpoint at regular intervals and stores the data for querying, alerting, and visualization.
- We can also write custom exporters.

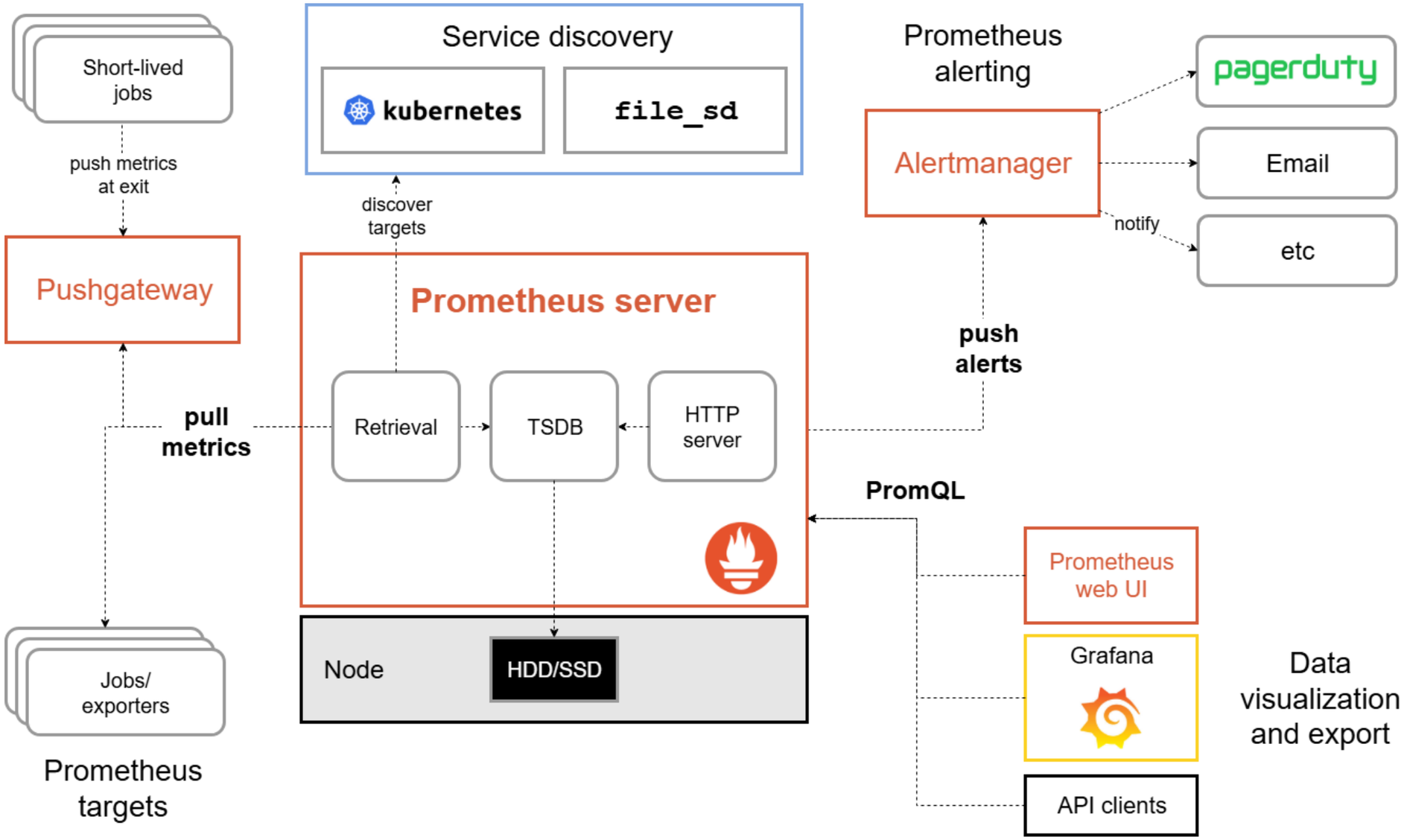
Specific Use Cases of Prometheus Monitoring – Example Scenario

- **Problem:**
  - A server **runs out of memory** → kills a **database-sync container** → **database pods fail** → authentication service stops → users see login errors.
- **Without Monitoring:**
  - Engineers manually trace failures backward — slow and inefficient.
- **With Prometheus:**
  - Continuously monitors memory usage, container/pod status, disk space, and network load.
  - Triggers alerts via Alertmanager when thresholds are exceeded (e.g., memory > 70%, storage > 50%).
  - Enables early detection, root-cause analysis, and prevention of cascading failures.
- **Other Scenarios:**
  - Disk usage alerts: prevent Elasticsearch from hitting storage limits.
  - Network spikes detection: identify misbehaving services causing slowdowns.

Specific Use Cases of Prometheus Monitoring

- Kubernetes Monitoring: Track pod resource usage, node health, cluster performance.
- Server Monitoring: Monitor CPU, memory, disk, and network stats on Linux servers.
- Application Performance Monitoring: Measure API latency, error rates, throughput.
- Database Monitoring: Observe query performance and connections (e.g., MySQL Exporter).
- Alerting: Trigger alerts when thresholds (like high CPU usage) are exceeded.
- Container Monitoring: Collect metrics from Docker containers using cAdvisor.
- Microservices Monitoring: Track inter-service communication and request success rates.

Prometheus Architecture



Prometheus Architecture

Prometheus Server

- Handles all monitoring, storage, and query operations.

- Consists of:
  1. Retrieval Worker: Pulls metrics from targets (applications, servers, exporters, jobs).
  2. Time-Series Database (TSDB): Stores collected metrics on disk (HDD/SSD).
  3. HTTP Server / Web API: Allows queries and visualizations via Prometheus Web UI or Grafana.

Prometheus Architecture

Targets and Metrics

- Targets: Entities being monitored, e.g.:
  - Linux/Windows servers
  - Applications/services (database, web apps, microservices)
  - Exporters or jobs that expose metrics
- Metrics: Specific measurable units for targets, e.g.:
  - Server: CPU usage, memory usage, disk space
  - Application: Request count, exceptions, request duration

Prometheus Architecture

Service Discovery & Metrics Flow

- Service Discovery: Automatically identifies targets via Kubernetes, file_sd, or other mechanisms.
- Metric Flow:
  1. Prometheus pulls metrics from targets.
  2. Data is stored in TSDB.
  3. Metrics are queried via HTTP server or visualized in dashboards.

Pushgateway: Handles short-lived jobs that push metrics on exit.

Prometheus Architecture

Alerting

- Alertmanager: Receives alerts from Prometheus based on rules.
- Sends notifications via: Email, PagerDuty, Slack, etc.
- Example: Memory > 70% triggers an alert before server crashes.

Prometheus Architecture

Visualization

- Prometheus Web UI: Basic query and visualization interface.
- Grafana: Advanced dashboards, charts, and alerts for metrics.
- API Clients: External systems can query Prometheus for integration.

Prometheus Architecture

Workflow Summary (End-to-End)

1. Targets expose metrics → Linux server, applications, exporters.
2. Prometheus server retrieves metrics → stores in TSDB.
3. Alerts are triggered → sent to Alertmanager.
4. Visualization & Querying → Prometheus UI, Grafana, API clients.
5. Pushgateway handles short-lived jobs → metrics pushed at job exit.

How Prometheus Collects Metrics from Targets?

1. The Pull Mechanism

- Prometheus pulls metrics from targets via an HTTP endpoint — http://<target>:<port>/metrics
- For this to work:
    1. The target must expose the /metrics endpoint.
    2. Data at /metrics must be in Prometheus' text-based format (key-value pairs).

How Prometheus Collects Metrics from Targets?

2. Role of Exporters

- Many systems (like Linux servers, MySQL, etc.) don't natively expose /metrics.
- Prometheus uses Exporters — small programs that:
  - Collect metrics from a system or app.
  - Convert them into Prometheus-readable format.
  - Expose them on their own /metrics endpoint.

Examples of Exporters:

- Node Exporter: for Linux servers (CPU, memory, disk, network).
- MySQL Exporter: for MySQL database metrics.
- cAdvisor: for Docker container metrics.
- Blackbox Exporter: for probing endpoints like HTTP, DNS, etc.
- windows_exporter: for Windows Server
- Prometheus-Flask-Exporter: for Flask webapp

Exporters are available as binaries or Docker images, often deployed as sidecar containers in Kubernetes.

How Prometheus Collects Metrics from Targets?

3. Monitoring Custom Applications

- Developers can instrument their applications using Prometheus client libraries:
  - Languages: Python, Java, Node.js, Go, etc.
  - Example
    - Python: We can use the prometheus_client library to instrument a Python application.
    - Java: The prometheus-java-client library is used for Java applications.
    - Node.js: For Node.js, you would use the prom-client library.
    - Go: The prometheus/client_golang library is commonly used in Go applications.
- These libraries allow you to:
  - Define and expose custom metrics (e.g., request count, latency, exceptions).
  - Serve them at /metrics for Prometheus to scrape.

This allows infrastructure teams to monitor application-level metrics alongside system metrics.

How Prometheus Collects Metrics from Targets?

4. Pull Mechanism – A Key Advantage

Most monitoring systems (like CloudWatch, New Relic) use a **push model**, where services send metrics to a central collector.

Aspect	Push Model	Prometheus Pull Model
Data flow	Services push metrics	Prometheus pulls metrics
Network load	High (many push requests)	Controlled (scheduled scrapes)
Visibility	Hard to detect service failures	Easy to detect if service is down
Scalability	Requires agents or daemons	Only needs /metrics endpoint

How Prometheus Collects Metrics from Targets?

4. Pull Mechanism – A Key Advantage

Most monitoring systems (like CloudWatch, New Relic) use a push model, where services send metrics to a central collector.

Aspect	Push Model	Prometheus Pull Model
Data flow	Services push metrics	Prometheus pulls metrics
Network load	High (many push requests)	Controlled (scheduled scrapes)
Visibility	Hard to detect service failures	Easy to detect if service is down
Scalability	Requires agents or daemons	Only needs /metrics endpoint

Benefits of Pull Approach:

- Reduces network overhead.
- Easier to detect target availability.
- Multiple Prometheus servers can scrape the same target.

How Prometheus Collects Metrics from Targets?

5. Pushgateway (for Short-lived Jobs)

- Some jobs (like batch jobs or backup scripts) run briefly — not long enough for Prometheus to scrape them.
- Such jobs use the Pushgateway, which:
  - Accepts metrics pushed by short-lived processes.
  - Makes them available to Prometheus temporarily.

Use Pushgateway sparingly — the pull model remains the default and preferred mechanism.

How Prometheus Collects Metrics from Targets?

Summary

Prometheus collects data primarily by scraping /metrics endpoints from targets. Exporters and client libraries make it easy to expose these metrics, and the pull-based design ensures efficient, scalable, and fault-aware monitoring.

Configuring Prometheus

1. Purpose of Configuration

- Prometheus needs to know:
  - **Which targets** to scrape (monitor).
  - **How often** to scrape them.
  - **Where** to find alerting or aggregation rules.
- All these settings are defined in the file: **prometheus.yml**

Configuring Prometheus

2. Main Sections in prometheus.yml

```
global:
  scrape_interval: 15s        # How often to scrape metrics (default 15s)
  evaluation_interval: 15s    # How often to evaluate rules
rule_files:
  - "rules.yml"
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

Recording rules: 1. Aggregate metrics for faster queries. 2. Alerting rules: Define conditions to trigger alerts (e.g., CPU > 80%).

job_name: Logical name for a monitoring task.
static_configs: Lists fixed endpoints to scrape.
Default /metrics path is used unless overridden.
Here, Prometheus scrapes its own metrics from localhost:9090/metrics.

Configuring Prometheus

3. Adding Custom Targets

We can add more jobs to monitor other servers or applications:

```
scrape_configs:
  - job_name: "linux_server"
    scrape_interval: 10s
    static_configs:
      - targets: ["192.168.1.10:9100"]   # Node exporter endpoint
  - job_name: "mysql"
    static_configs:
      - targets: ["192.168.1.11:9104"]   # MySQL exporter endpoint
```

- Each job can have its own scrape interval.
- Targets can be discovered dynamically via service discovery (e.g., Kubernetes, AWS EC2).

Configuring Prometheus

4. Summary

- prometheus.yml is the heart of Prometheus configuration.
- Defines what to monitor, how often, and which rules to apply.
- Easy to extend by adding new jobs or dynamic service discovery.

Alertmanager – Triggering Alerts

- **Purpose:** Handles alerts generated by the Prometheus server.
- **Workflow:**
  - Prometheus evaluates alert rules defined in rules.yml.
  - If a rule condition is met, an **alert** is triggered.
  - The **Alertmanager** receives the alert and routes it through configured channels.
- **Notification Channels:**
  - Email
  - Slack
  - PagerDuty
  - Webhooks / Other integrations

Example: If CPU usage > 80% for 2 minutes, Alertmanager sends a warning to Slack.

Prometheus Data Storage

- **Local Storage:**
  - Prometheus stores metrics in a **local on-disk Time Series Database (TSDB)**.
  - Data is saved in a **custom format**, optimized for fast writes and queries.
- **Remote Storage (Optional):**
  - Can integrate with long-term storage like **Thanos**, **Cortex**, or **VictoriaMetrics**.
- **Data Access:**
  - Prometheus exposes a **HTTP-based API** to query stored metrics.
  - Metrics are retrieved using **PromQL (Prometheus Query Language)**.

PromQL – Querying Metrics

- **PromQL** allows querying, filtering, and aggregating metrics.
- Queries can be run via:
  - **Prometheus UI** (built-in dashboard)
  - **Grafana** (for advanced visualization)

```
# Get all HTTP status codes except 4xx
http_requests_total{status!~"4.."}

# Calculate 5-minute rate of requests
rate(http_requests_total[5m])
```

Grafana uses PromQL under the hood to visualize metrics in dashboards.

Key Characteristics – Advantages & Disadvantages

- **Advantages:**
  - **Standalone and self-contained:** Works even if other systems fail.
  - **No external dependencies:** Doesn't rely on network storage or remote services.
  - **Fast setup:** Single-node Prometheus can be deployed easily.
  - **High reliability:** Keeps collecting metrics during outages.

Disadvantages:

- **Difficult to scale:** Managing multiple Prometheus servers and aggregating data is complex.
- **Storage limits:** A single Prometheus instance can handle only a limited amount of metrics.
- **Configuration overhead:** Needs careful setup to avoid redundant or irrelevant data collection.

Downloading Prometheus

- Main Page: https://prometheus.io/download/
- For windows, download:
  - https://github.com/prometheus/prometheus/releases/download/v3.5.0/prometheus-3.5.0.windows-amd64.zip

Configuring Prometheus

- The Prometheus download comes with a sample configuration in a file called prometheus.yml that is a good place to get started.

```
global:
  scrape_interval:     15s
  evaluation_interval: 15s


rule_files:
  # - "first.rules"
  # - "second.rules"


scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
```

- There are three blocks of configuration in the example configuration file: global, rule_files, and scrape_configs.

Configuring Prometheus

- The global block controls the Prometheus server's global configuration.
- We have two options present.
  - The first, scrape_interval, controls how often Prometheus will scrape targets. We can override this for individual targets. In this case, the global setting is to scrape every 15 seconds.
  - The evaluation_interval option controls how often Prometheus will evaluate rules. Prometheus uses rules to create new time series and to generate alerts.

Configuring Prometheus

- The rule_files block specifies the location of any rules we want the Prometheus server to load. For now we've got no rules.

Configuring Prometheus

- The last block, scrape_configs, controls what resources Prometheus monitors.
- Since Prometheus also exposes data about itself as an HTTP endpoint, it can scrape and monitor its own health.
- In the default configuration, there is a single job, called prometheus, which scrapes the time series data exposed by the Prometheus server.
- The job contains a single, statically configured target, the localhost on port 9090.
- Prometheus expects metrics to be available on targets on a path of /metrics. So this default job is scraping via the URL: http://localhost:9090/metrics.
- The time series data returned will detail the state and performance of the Prometheus server.

Starting Prometheus

- To start Prometheus with our newly created configuration file, change to the directory containing the Prometheus binary and run:
  - ./prometheus --config.file=prometheus.yml

- Prometheus should start up. We should also be able to browse to a status page about itself at http://localhost:9090. Give it about 30 seconds to collect data about itself from its own HTTP metrics endpoint.
- We can also verify that Prometheus is serving metrics about itself by navigating to its own metrics endpoint: http://localhost:9090/metrics.

Using the Expression Browser

- Let us try looking at some data that Prometheus has collected about itself.
- To use Prometheus's built-in expression browser, navigate to http://localhost:9090/graph and choose the "Table" view within the "Graph" tab.
- As we can gather from http://localhost:9090/metrics, one metric that Prometheus exports about itself is called promhttp_metric_handler_requests_total (the total number of /metrics requests the Prometheus server has served).
- Go ahead and enter this into the expression console:
  - promhttp_metric_handler_requests_total
- This should return a number of different time series (along with the latest value recorded for each), all with the metric name promhttp_metric_handler_requests_total, but with different labels. These labels designate different request statuses.

Using the Graphing Interface

- To graph expressions, navigate to http://localhost:9090/graph and use the "Graph" tab.
- For example, enter the following expression to graph the per-second HTTP request rate returning status code 200 happening in the self-scraped Prometheus:
  - rate(promhttp_metric_handler_requests_total{code="200"}[1m])
- We can experiment with the graph range parameters and other settings.

Monitoring other Targets

- Collecting metrics from Prometheus alone isn't a great representation of Prometheus' capabilities.
- To get a better sense of what Prometheus can do, we should explore documentation about other exporters.

When does Prometheus Fit?

- Prometheus works well for recording any purely numeric time series.
- It fits both machine-centric monitoring as well as monitoring of highly dynamic service-oriented architectures.
- In a world of microservices, its support for multi-dimensional data collection and querying is a particular strength.
- Prometheus is designed for reliability, to be the system you go to during an outage to allow you to quickly diagnose problems.
- Each Prometheus server is standalone, not depending on network storage or other remote services.
- We can rely on it when other parts of your infrastructure are broken, and we do not need to setup extensive infrastructure to use it.

When does Prometheus not Fit?

- Prometheus values reliability. You can always view what statistics are available about your system, even under failure conditions.
- If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice, as the collected data will likely not be detailed and complete enough. Prometheus might miss some data because it scrapes metrics at intervals and doesn't guarantee capturing every single event. This makes it less suitable for things like billing, where we need complete and detailed logs.
- In such a case, we would be best off using some other system to collect and analyze the data for billing, and Prometheus for the rest of your monitoring.

Disclaimer

- Content of this presentation is not original and it has been prepared from various sources for teaching purpose.